

# Guía de supervivencia para el parcial de SO

Virtualización de CPU + Memoria Virtual · Basada en los prácticos 2020 y el resumen de OSTEP. Son **claves y trucos**, no resoluciones: la promesa del profe es que quien resuelve la guía solo, aprueba. Usá esto para chequear que no te olvidás nada.

Método

Mecanismos

Políticas

API memoria

Traducción

Paginación

Verdadero/Falso

Plan

## 0. Reglas de oro para el examen

- **Escribí la fórmula antes de los números.** Cuesta puntos hacer bien la cuenta con la fórmula equivocada.
- **Declará tus convenciones** (desempates en RR, si una llegada entra antes que el proceso desalojado, KiB vs KB). Si es coherente, se acepta.
- **Dibujá siempre** la línea de tiempo / la tabla de páginas / el split de bits. La mayoría de los errores son de bookkeeping, no de concepto.
- En V/F la nota está en el "**porque**". Una frase con el mecanismo (trap, bit de validez, COW...) vale más que un "verdadero".
- Potencias:  $4 \text{ KiB} = 2^{12}$ ,  $4 \text{ MiB} = 2^{22}$ ,  $4 \text{ GiB} = 2^{32}$ . Hex: cada dígito = 4 bits.

## 1. Virtualización de CPU: mecanismos

### Tiempos: real, user, sys (Ej. 1-3)

**real** = reloj de pared · **user+sys** = CPU efectivamente consumida.

**real** > **user**: el proceso esperó (cola de ready por falta de núcleos, IO, otro proceso).

**real**  $\approx$  **user**: un solo hilo CPU-bound con un núcleo para él solo.

**real** < **user**: varios hilos corriendo *en paralelo* en varios núcleos (dgemm).

**Truco núcleos:** con  $n$  procesos CPU-bound idénticos y  $c$  núcleos,  $\text{real} \approx \text{user} \cdot n/c$  cuando  $n > c$ , y  $\text{real} \approx \text{user}$  cuando  $n \leq c$ . Buscá la corrida donde el real empieza a crecer: ahí superaste los núcleos. Si **una sola instancia** tiene  $\text{real} > \text{user}$ , algo *externo* le está robando CPU (otra carga en el sistema). Anotá eso en "Descripción".

Sí,  $\text{user+sys} < \text{real}$  es posible (IO o espera).  $\text{user+sys} > \text{real}$  también, con hilos.

## fork / exec / wait (Ej. 4-9)

- **fork** copia el espacio de direcciones (lógicamente; en la práctica COW). Padre e hijo tienen **x independientes**: cada uno cambia la suya. Si **x** está en un registro, da igual: los registros también se copian/guardan por proceso.
- Retorno: hijo  $\rightarrow 0$ , padre  $\rightarrow$  PID del hijo, error  $\rightarrow -1$ . Que el hijo vea 0 es una *convención*, no consecuencia de "nadie tiene PID 0".
- **exec** reemplaza código, datos, heap y stack; **no retorna** salvo error. Lo que sigue a `execv()` corre solo si falló (Ej. 6: el "a" no se imprime).
- **Contar "a" con forks (Ej. 5)**: imprimí, contá procesos que hay en cada printf. Con  $n$  forks:  
 $1+2+4+\dots+2^n = 2^{n+1}-1$ . **Trampa**: si stdout va a un archivo/pipe, el buffer (full buffering) se duplica con el fork y el resultado cambia; ahí cada uno de los  $2^n$  procesos finales vuelca todo su historial.
- **Ej. 7**: mirá qué hace `argv[argc-1]=NULL` con la lista de argumentos y quién hace el `execvp` (y quién termina antes). Simulá con `./prog echo hola mundo`.
- **Ej. 8 (dup)**: los descriptors se heredan por fork y **sobreviven a exec**; `open / dup` devuelven el **menor fd libre**. Por eso el shell hace `fork`  $\rightarrow$  (redirige fds en el hijo)  $\rightarrow$  `exec`. Separar fork y exec es lo que permite redirigir sin tocar al padre: no es mal diseño, es la clave.
- **Fork bomb**: duplica procesos exponencialmente hasta llenar la tabla de procesos/memoria. Mitigación: límites por usuario (`ulimit -u`), `cgroups` (`pids.max`), cuotas.
- **Zombie**: hijo terminado cuyo padre no hizo `wait()`; sigue en la tabla de procesos guardando el exit status. Si el padre muere, los hijos son adoptados por `init` (no mueren).

## Limited Direct Execution (Ej. 10-12)

- El proceso corre **directo en la CPU** (eficiencia) pero en **modo usuario** (control). Instrucciones privilegiadas (I/O, modificar base/bounds, tabla de traps, CR3)  $\rightarrow$  excepción si las usa.
- Sin timer interrupt no hay control: un `while(1)` se queda con la CPU. La **tabla de traps** se configura al boot en modo kernel.
- **Dos guardados de registros en un cambio de contexto por timer**: (1) el **hardware** guarda registros de usuario en el kernel stack al entrar al trap; (2) el **software** (`swtch`) guarda los registros del kernel en la struct context del proceso.
- **trapframe xv6 (Ej. 11)**: el **hardware** apila `ss`, `esp` (solo si cambia de privilegio), `eflags`, `cs`, `eip` (+ error code en algunas excepciones). El **software** apila lo demás: `trapno` (`vectors.S`), `ds`, `es`, `fs`, `gs` y los registros generales (`pushal`) en `alltraps`.
- **Ej. 10, quitar una flecha del diagrama de estados**: sin *Ready*  $\rightarrow$  *Running*: nada se ejecuta. Sin *Running*  $\rightarrow$  *Ready*: no hay apropiación, un proceso monopoliza la CPU. Sin *Running*  $\rightarrow$  *Blocked*: el proceso hace busy-wait en IO desperdiciando CPU. Sin *Blocked*  $\rightarrow$  *Ready*: los procesos que piden IO nunca despiertan.

## 2. Políticas de planificación

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

$$T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$$

Ambos **desde la llegada**, no desde 0. Es el error #1.

| Política | Idea                                                      | Bueno en                                   | Malo en                                         |
|----------|-----------------------------------------------------------|--------------------------------------------|-------------------------------------------------|
| FCFS     | orden de llegada                                          | simple                                     | efecto convoy                                   |
| SJF      | el más corto primero (no apropiativo)                     | turnaround (óptimo si todos llegan juntos) | si el largo llega primero; response; starvation |
| STCF     | SJF apropiativo: corre el de menor tiempo <i>restante</i> | turnaround                                 | response                                        |
| RR (Q)   | rota cada Q                                               | response                                   | turnaround (Q chico + costo de switch)          |
| MLFQ     | colas con prioridad, aprende de la historia               | ambos, sin saber $T_{\text{CPU}}$          | gaming, starvation sin boost                    |

- **Ej. 15:** lotes = FCFS, SJF, STCF. Interactivas = RR, MLFQ. **Necesitan conocer el TCPU de antemano:** SJF y STCF. FCFS, RR y MLFQ no (MLFQ lo estima con la historia).
- **Ej. 13:** con llegada 0 para todos, FCFS orden A,B,C; SJF orden C,B,A. Comparalos: el turnaround promedio baja mucho, el response de FCFS = tiempo de espera.
- **Ej. 14 (STCF/RR):** en cada instante en que llega alguien, reevalúa. Anotá la regla de desempate de RR (lo habitual: el que llega entra a la cola *antes* que el que se desaloja en ese mismo instante).
- **Ej. 16 (IO):** mientras un proceso hace IO está **bloqueado**, no ocupa la CPU ni está en la cola de ready. Al terminar el IO vuelve al final de la cola. La CPU no queda ociosa si hay otros en ready. Marcá los tramos de IO con otra trama en el diagrama.
- **Ej. 17 (MLFQ, 4 colas Q=1,2,4,8):** reglas: (1) mayor prioridad corre primero; (2) igual prioridad → RR; (3) trabajo nuevo entra en la cola **más alta**; (4) si agota su quantum en la cola, **baja**; si suelta la CPU antes, se queda (versión mejorada: se acumula el tiempo total por nivel y no importa cuántas veces hizo yield); (5) boost periódico sube todos a la cima. Un trabajo nuevo **apropia** al que está en cola inferior.

**Chequeo rápido:** la suma de los ancho de tramos en tu diagrama debe dar la suma de los TCPU (si no hay huecos). Si hay hueco, ¿llegó alguien más tarde o estaban todos en IO?

### 3. API de memoria

- **Ej. 1:** globales ( `a` ) → segmento de **datos** (`.data / .bss`), no `stack` ni `heap`. Un global inicializado a cero suele ir a `.bss` (no ocupa espacio en el ejecutable). Locales ( `i` , `argc` , `argv` ) → **stack**.  
`register int s` → un **registro**, no tiene dirección. `b` (el puntero) vive en el **stack** y **apunta al heap**. El código va en el segmento de código.
- **Ej. 2, bugs a cazar:** `gets` no tiene límite (buffer overflow; usar `fgets` ).  
`malloc(strlen(s))` olvida el `+1` del `'\0'` . `d = strdup(s)` tras un `malloc` pisa el puntero: **memory leak**. `malloc(16)` son 16 *bytes*, no 16 *ints*: `a[15]` es out-of-bounds. Además: falta el `;` y `sizeof` .
- **malloc/free (Ej. 3):** son **funciones de biblioteca**, no syscalls. Solo *a veces* llaman a `brk/sbrk` o `mmap` (cuando se les acaba el `heap`) y `free` a veces a `munmap` . El tiempo **no** es proporcional a `x` (depende de la lista de libres).

### 4. Traducción de direcciones

#### Base + bounds

- Física = base + virtual, y antes se verifica `virtual < bounds` . **Con bounds=256 la dirección 256 ya es inválida.**
- **Ej. 4:** tené en cuenta que **cada fetch de instrucción también es un acceso a memoria** (a base + PC), además de los loads/stores. Recorré las 5 instrucciones y fijate qué pasa cuando `%ebx` se duplica.
- **Fragmentación interna:** espacio desperdiciado *dentro* de la región asignada (el hueco `stack-heap` sin usar en cada proceso). **Externa:** hay memoria libre total suficiente pero en huecos no contiguos chicos.
- **Relocalización estática:** el loader reescribe direcciones al cargar (sin protección, no se puede mover después). **Dinámica:** el hardware suma base en cada acceso (protege y permite mover).
- Modificar base/bounds es **privilegiado**. Hardware: un par por CPU; el SO guarda/restaura los valores de cada proceso (en su PCB) en el context switch.

#### Segmentos con páginas (Ej. 8) y espacio libre (Ej. 9)

**Ej. 8:** redondeá **cada segmento hacia arriba** a múltiplo de página (no se mezclan tipos), sumá páginas y compará con las que hay (`65536 / tamaño de página`). Comparé el caso 4 KiB vs 512 B: la fragmentación interna cambia todo.

- **First fit:** primer hueco que alcanza. **Best:** el más chico que alcanza. **Worst:** el más grande. **Next:** como first pero *continuando desde donde quedó* la última búsqueda.
- **Trampa:** tras cada asignación el hueco **se achica** (queda el resto), y ese resto puede servir a la siguiente solicitud. Actualizá la lista en cada paso.

## 5. Paginación

**Split:**  $VPN = \text{dir} \gg 12 \cdot \text{offset} = \text{dir} \& 0xFFF$ . Física =  $\text{frame} \cdot 4096 + \text{offset}$ .

**i386 (10,10,12):**  $PDX = \text{dir} \gg 22 \cdot PTX = (\text{dir} \gg 12) \& 0x3FF \cdot \text{offset} = \text{dir} \& 0xFFF$ .

**Siempre** mirá el bit de validez/presente antes: si es 0 → **page fault**, no hay dirección física.

- **Ej. 10 (EAT):** acierto  $TLB = t_{TLB} + t_{mem}$ . Fallo con tabla lineal =  $t_{TLB} + t_{mem}(\text{tabla}) + t_{mem}(\text{dato})$ .  
EAT =  $0,95 \cdot \text{acierto} + 0,05 \cdot \text{fallo}$ . Da 136 ns.
- **Ej. 11 (TLB):** falla en cada iteración si el paso salta de página en cada acceso ( $M \cdot 4 \text{ B} \geq 4096$ ) y se tocan más de 64 páginas distintas (con LRU/FIFO se expulsan justo antes de reusarse). Si el ciclo se repite y todo entra en 64 entradas, tras la primera pasada solo quedan aciertos.
- **Ej. 12:** 16 páginas virtuales  $\cdot 4 \text{ KiB} = 16$  bits virtuales; frames de 3 bits → 15 bits físicos. Para el mapeo inverso, **no es único**: dos páginas virtuales pueden apuntar al mismo frame (ej.: páginas 4 y 10 → frame 100). Dá todas las válidas.
- **Ej. 13:** separá los hex en (PDX, PTX, offset) con las máscaras de arriba, seguí CR3 → PD[PDX] → PT[PTX] → frame + offset.
- **Ej. 14:** una PT (1024 entradas de 4 B) ocupa 4 KiB y mapea 4 MiB. (a)  $32 \text{ MiB} = 8 \text{ PT} + 1 \text{ PD} = 36 \text{ KiB}$ . (b)  $1024 \text{ PT} + 1 \text{ PD} = 4 \text{ MiB} + 4 \text{ KiB}$ . (c) Una tabla lineal de  $2^{20}$  entradas ocupa **siempre 4 MiB**: con todo mapeado 2 niveles cuesta 4 KiB de más, pero con espacios *ralos* (lo normal) 2 niveles ahorra muchísimo. (d) 0–4 MiB: PD[0] → 1 PT con 1024 entradas; 8 MiB: PD[2] → PT con 8 entradas que apuntan a frames  $128 \text{ MiB} / 4 \text{ KiB} = 32768 \dots$ ; el resto, no presente.
- **Ej. 15:** las tablas también ocupan frames físicos; mapear los 4 GiB completos consume 1025 frames de tablas, así que no queda para el resto. Máximo  $\approx 2^{20} - 1025$  páginas de datos.
- **Ej. 16 (PAE):** entradas de 8 bytes (para direccionar 36 bits) → 512 entradas por tabla, 3 niveles (2,9,9,12) con un PDPT; cada proceso sigue viendo 4 GiB pero el SO reparte frames de 64 GiB entre todos.
- **Ej. 18 (recursive page table):** PD[1023] = el propio PD, entonces al caminar dos niveles se "pierde" uno.  $0xFFC00000 \rightarrow PDX=1023, PTX=0 \rightarrow$  apunta a la **primera page table**.  
 $0xFFFE0000 \rightarrow PTX=992 \rightarrow$  la page table n° 992.  $0xFFFFF000 \rightarrow PDX=PTX=1023 \rightarrow$  el **page directory** mismo. Sirve para que el kernel lea/escriba las page tables (y el PD) con direcciones virtuales normales, sin mapeos aparte.

**Ej. 10: paginación como herramienta (una línea de mapeo por cada uno)**

**Ej. 17. paginacion como mermatenta (una linea de mecanismo por cada una)**

| Técnica             | Mecanismo                                                                                     |
|---------------------|-----------------------------------------------------------------------------------------------|
| NULL falla          | La página 0 no se mapea (inválida) → page fault → SIGSEGV.                                    |
| Swap                | Bit present=0 + dirección en disco; el fault trae la página y actualiza la PTE.               |
| Demand paging       | El ejecutable se mapea como inválido; se carga cada página al primer fault.                   |
| Stack auto-growing  | Fault cerca del tope del stack (guard) → el kernel asigna un frame nuevo.                     |
| Stack no ejecutable | Bit NX/permisos en la PTE; ejecutar ahí → fault.                                              |
| mmap                | Rango virtual respaldado por archivo; se lee en el fault, se escribe de vuelta si está dirty. |
| COW en fork         | Hijo comparte frames con padre marcados solo-lectura; al escribir, el fault copia la página.  |
| sbrk/malloc barato  | Solo sube el límite del heap; los frames se asignan al primer acceso.                         |
| calloc barato       | Mapear páginas al "frame de ceros" compartido con COW: no hace falta memset.                  |
| Código compartido   | Mismos frames de solo-lectura mapeados en varios procesos (libs, kernel).                     |
| Memoria compartida  | El mismo frame en las tablas de dos procesos con permiso de escritura.                        |

## 6. Verdadero/Falso: veredictos y el "porque"

Intentá primero sin mirar. Cada línea: veredicto + palabra clave.

### Ej. 12 (mecanismos)

- (a) **V** · espera de IO/cola. (b) **F** · cada proceso tiene su espacio virtual, misma dirección → distintos frames. (c) **F/matiz** · en el trap se guardan los de usuario; en el switch, los callee-saved del kernel. (d) **F** · las privilegiadas dan trap. (e) **V** · el scheduler puede reelegir al mismo. (f) **F** · es convención de la API. (g) **V**. (h) **F** · quedan huérfanos, adopta init. (i) **F** · hay `wait` /exit status, pipes, archivos. (j) **F** · corre si `execv` falla. (k) **V** · zombie.

### Ej. 18 (políticas)

- (a) **F** · el control vuelve por bloqueo, yield, exit o interrupt. (b) **F** · SJF es óptimo solo con llegada simultánea; con llegadas distintas y no apropiativo no hay garantía. (c) **V**. (d) **V** · sin boost, los CPU-bound quedan abajo. (e) **V** · contabilizar el tiempo total por nivel evita el "yield antes del quantum".

### Ej. 3 y 7 (memoria)

- malloc syscall **F**; "siempre llama syscall" **F**; "a veces" **V**; free idem; tiempo  $\propto x$  **F**. Modificar base/bounds privilegiado **V**; un par "por proceso" **V** como estado guardado (no como registros físicos).

### Ej. 17 (paginación)

- (a) **V**. (b) **F** · puede ser al revés (PAE) o iguales. (c) **V**. (d) **V** · compartida. (e) **F** · una virtual apunta a un frame. (f) **V/matiz** · tamaño del espacio sí, pero no todo está mapeado y el kernel usa parte. (g) **V** · compartir, COW, demand. (h) **F**. (i) **F** · uno por proceso (el kernel se replica). (j) **V** · libres. (k) **F** · COW. (l) **F** · con paginación activa todo pasa por la MMU. (m) **F** · posible pero no único. (n) **V** · el PD solo ocupa 4 KiB.

## 7. Plan de estudio (lo que dijo el profe)

---

1. Hoja en blanco, **hacé todos los ejercicios sin mirar**. Cronometrate.
2. Marcá los que dudaste; recién ahí volvé al libro (OSTEP 4–9 para CPU, 13–20 para memoria).
3. Repetí solo los marcados, y **explicalo en voz alta a un compañero**: si no lo podés explicar, no lo sabés.
4. Última pasada: fórmulas de esta guía y las tablas de V/F.

Hecho para estudiar en grupo. Corregí lo que veas mal y pasalo. ¡Éxitos!

